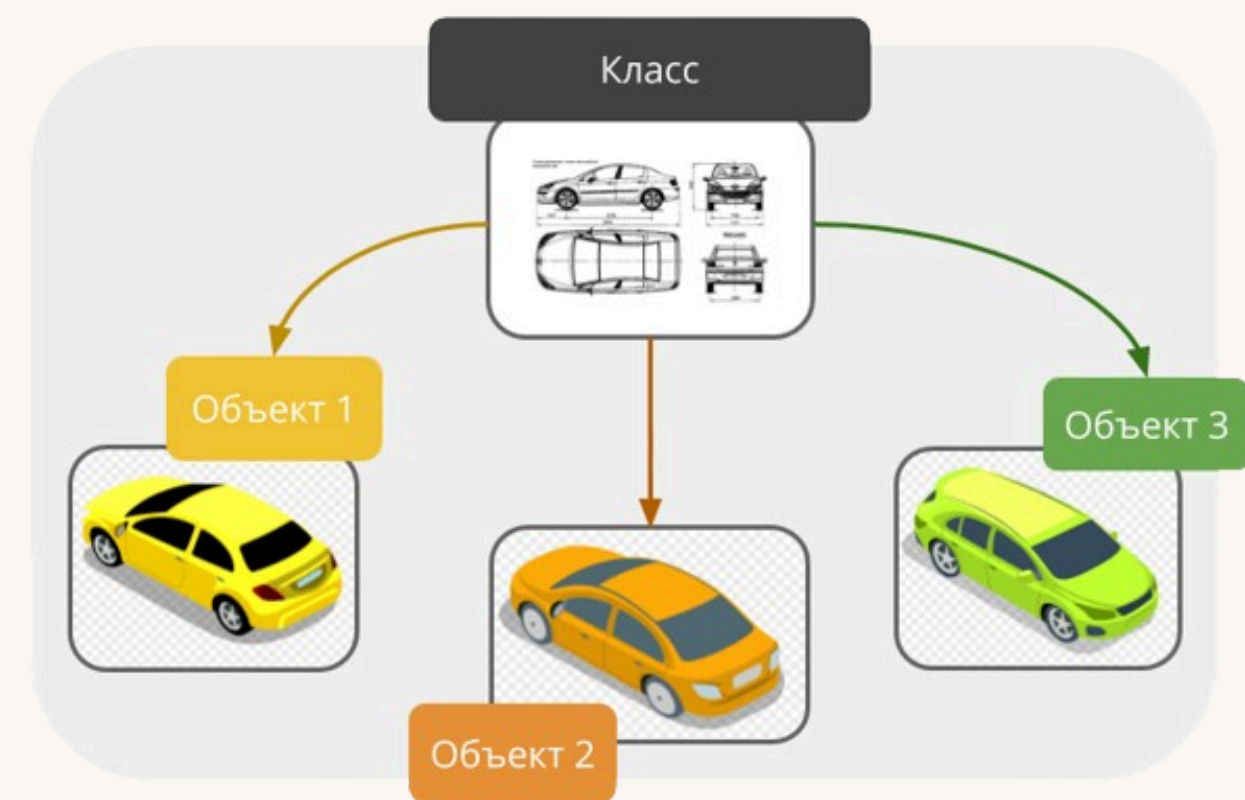


Введение в ООП



Абстракции





Что такое абстракция?

Абстракция – это создание маленькой модели большого объекта или его отражения.

Например, для приложения, которое отслеживает положение самолетов на карте мира, абстракция будет - рейс. Вы смотрите на карту, там летят самолеты - это будет положение самолета, номер рейса, модель, авиакомпания, откуда и куда летит.

Рассмотрим данную абстракцию для билетной кассы:

Это номер, время вылета и прилета, количество свободных мест, цена мест в экономе и бизнесе.

Для диспетчеров - рейс (тот же самый самолет который точно также летит) – это модель самолета, номер рейса, нужная ему длина полосы, назначенная полоса, время взлета (но не посадки, потому что для системы диспетчеров не так важно когда самолет сядет!), воздушный коридор, состояние самолета, загружены ли пассажиры, багаж и т.д.

Есть еще ряд других систем: таможня, управление авиаперсоналом, багажом и т.д.

И у всех этих систем абстракции одного объекта немного разные.





Что содержат в себе абстракции

Абстракции – содержат в себе **поля** и **методы**.

Поле - это не новый термин, это просто свойство, записанное в такой переменной (которая и называется полем, также называют атрибутом), внутри нашей абстракции. Например:

- **поля самолета:** производитель, год выпуска, модель, места, состояние, бортовой номер;
- **поля рейса:** самолет, места, откуда вылет, куда прилетает, время взлета, время посадки, экипаж;
- **поля билета:** номер рейса, номер места, время вылета, время посадки.

Методы - это действия над абстракцией или действие, которое абстракция может делать. По факту это обычные функции (но их принято называть методами), только они находятся в теле нашей абстракции.

Например:

- методы самолета - снять с полета, поставить на линию, заправить, разгрузить, погрузить;
- методы рейса - перенести, отменить, назначить экипаж;
- методы билета - распечатать, сдать, поменять, отменить.



Как выглядят абстракции

Абстракция	Самолет	Рейс	Билет
поля	• производитель • год выпуска • модель • места • состояние • бортовой номер	• самолет • места • откуда вылет • куда прилетает • время взлета • время посадки • экипаж	• номер рейса, • номер места, • время вылета, • время посадки.
методы	• снять с полета • поставить на линию • заправить • разгрузить • погрузить	• перенести • отменить • назначить экипаж	• распечатать, • сдать, поменять, • отменить.



UML Class Diagram (Формальный графический язык описания классов)

У каждой абстракции есть свои поля и методы. Эта схема, которую, вы видите, трехкомпонентный блок, где есть: Название, Поля, Методы

На самом деле такой формат взят не из головы.

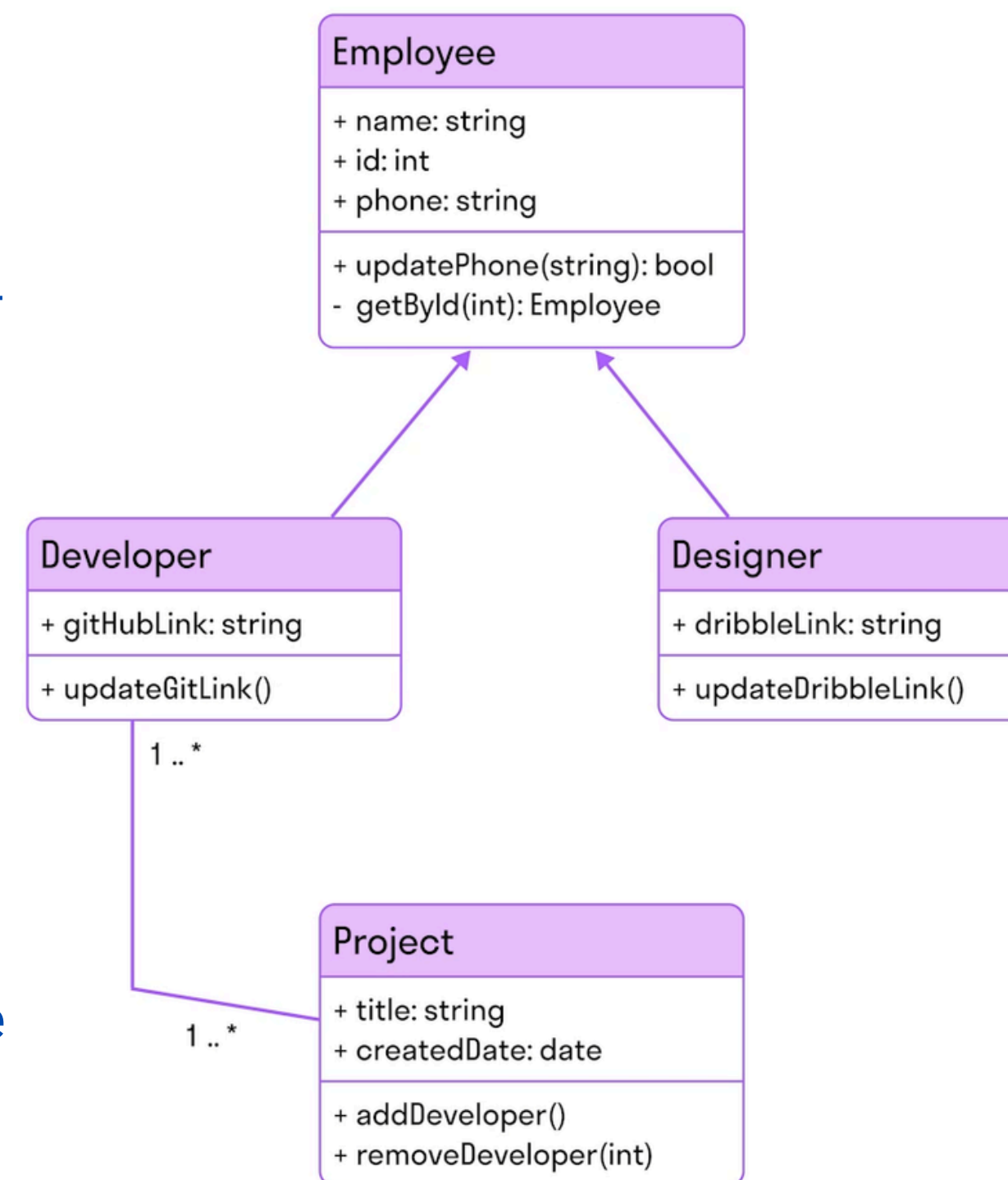
Это формат UML Class Diagram или диаграмма классов UML. А UML это Unified Modeling Language, то есть универсальный язык моделирования всего.

Это графическое представление набора абстракций. Между ними еще есть связи стрелочки.

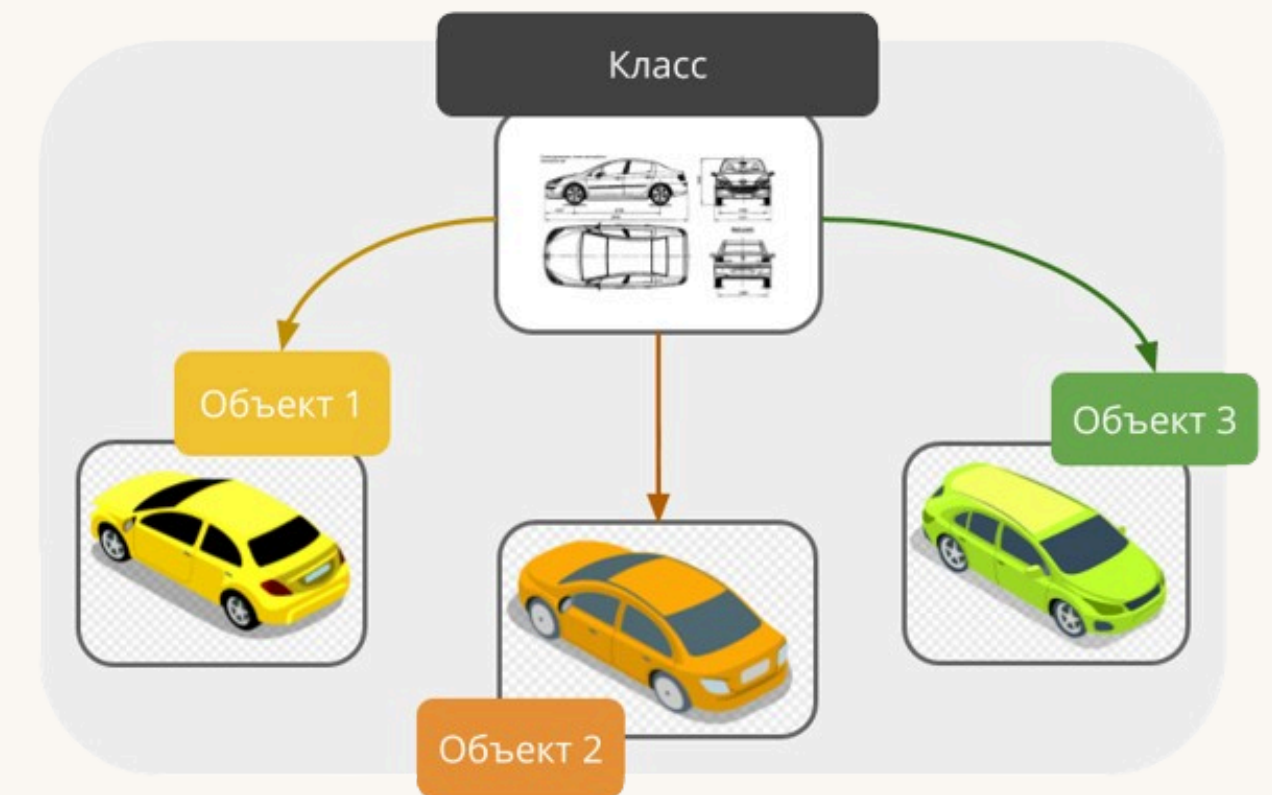
В данном примере у нас четыре абстракции некоторой информационной системы, где есть сотрудники, дизайнеры, разработчики и проекты.

У сотрудника есть имя, идентификационный номер, телефон и методы. Естественно, у деvelopepa, дизайнера и проекта есть тоже какие-то поля и методы.

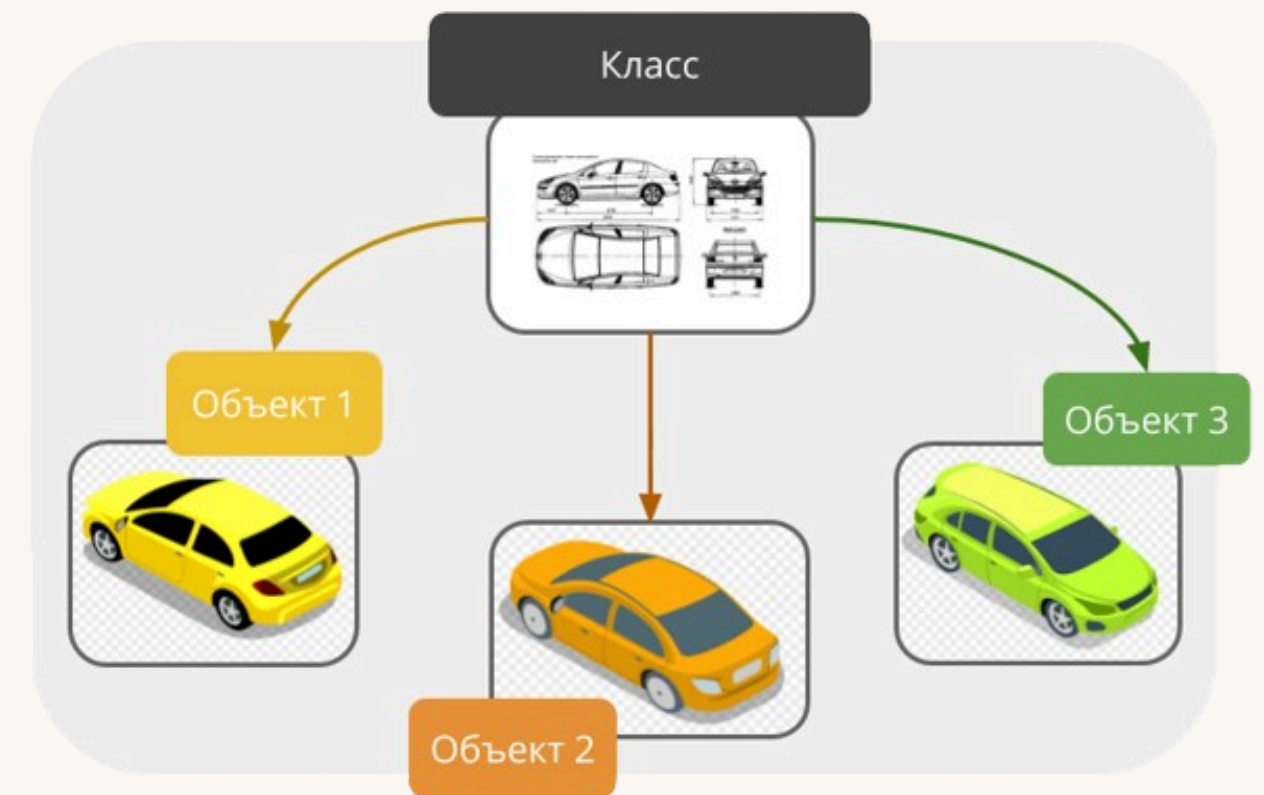
**если вы увидите или услышите термины, класс, модель, сущность имейте в виду что это может быть одно и то же.*



Практика. Составьте 2 примера абстракции с полями и методами. Имейте в виду, самые интересные на мой взгляд абстракции, необходимо будет реализовать в домашнем задании



Создание классов





Создание классов

Допустим мы хотим испечь кекс.
Создадим класс CakeForm, и несколько методов для него:

После чего мы создадим 2 экземпляра: **cake_1** и **cake_2** и вызовем написанные методы:

В результате работы (выпечки) мы получим следующий итог:

```
class CakeForm:

    def make_dough(self):
        return f"Мы замешиваем тесто"

    def make_form(self):
        return f"Мы выкладываем тесто в форму"

    def cook_cake(self, time=40):
        if time > 80:
            return f"За {time} минут кексик сгорит"
        return f"Мы выпекаем тесто {time} минут"
```

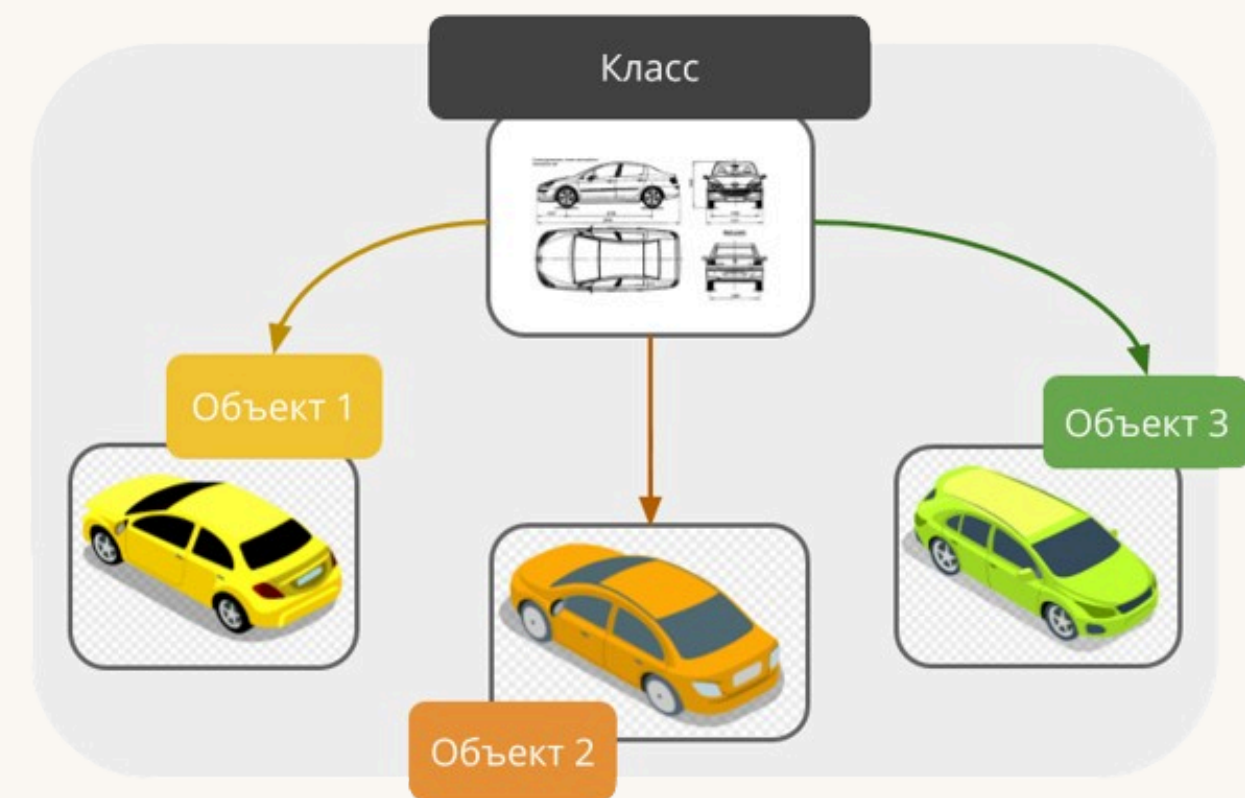
```
cake_1 = CakeForm()
cake_2 = CakeForm()

print(cake_1.make_dough())
print(cake_1.make_form())
print(cake_1.cook_cake())
print(cake_2.cook_cake(100))
```

Мы замешиваем тесто
Мы выкладываем тесто в форму
Мы выпекаем тесто 40 минут
За 100 минут кексик сгорит

Как видим у нас 2 экземпляра и 2й экземпляр - **cake_2** - для программы совершенно другой объект, т.к. при вызове метода cook_cake(), но с другим параметром который попал под определенное нами условие, мы получаем совершенно другой результат.

Инициализаторы классов





Что такое инициализаторы.

Инициализатор - это магический метод или дандер-метод. Он называется магическим, потому что он не вызывается программистом, а вызывается автоматически при создании нового экземпляра. Вы еще познакомитесь с подобными методами, их в питоне у каждого класса может быть много.

```
1 class CakeForm:
    2     def __init__(self):
    3         4 print("Инициализируюсь")
5 cake_1 = CakeForm()
  cake_2 = CakeForm()
```

1 - Класс
2 - Дандер метод
3 - Обязательный self
4 - Тело метода
5 - Создание экземпляров

>>> Инициализируюсь
>>> Инициализируюсь

Когда мы создаем экземпляр класса, у нас автоматически вызывается метод `__init__`. Эти нижние подчеркивания называются underscore, а дандер это двойные андерскоры. Методы которые окружены двойными андерсокорами называются дандер-методами.



Инициализаторы добавим уникальности экземплярам.

До этого наши экземпляры, просто были, они по факту были практически одинаковыми и мы не знали, из какого теста пекли наш кекс и в какой форме мы его выпекали, это были просто тесто и просто форма. Для того чтобы придать уникальности нашему экземпляру, добавим **поля** (их еще можно назвать атрибутами) в наш `__init__`:

```
class CakeForm:

    def __init__(self, form, dough):
        self.form = form
        self.dough = dough
        print(f"Я кекс в форме - {self.form}, из теста - {self.dough}!")
```

```
cake_1 = CakeForm("круглая", "мучное")
cake_2 = CakeForm("квадратная", "овсяное")
```

▼ ▼ ▼

Я кекс в форме - круглая, из теста - мучное!

Я кекс в форме - квадратная, из теста - овсяное!

У нас есть `cake_1` и `cake_2`, которым придаются соответствующие аргументы:

- форма;
- вид теста.

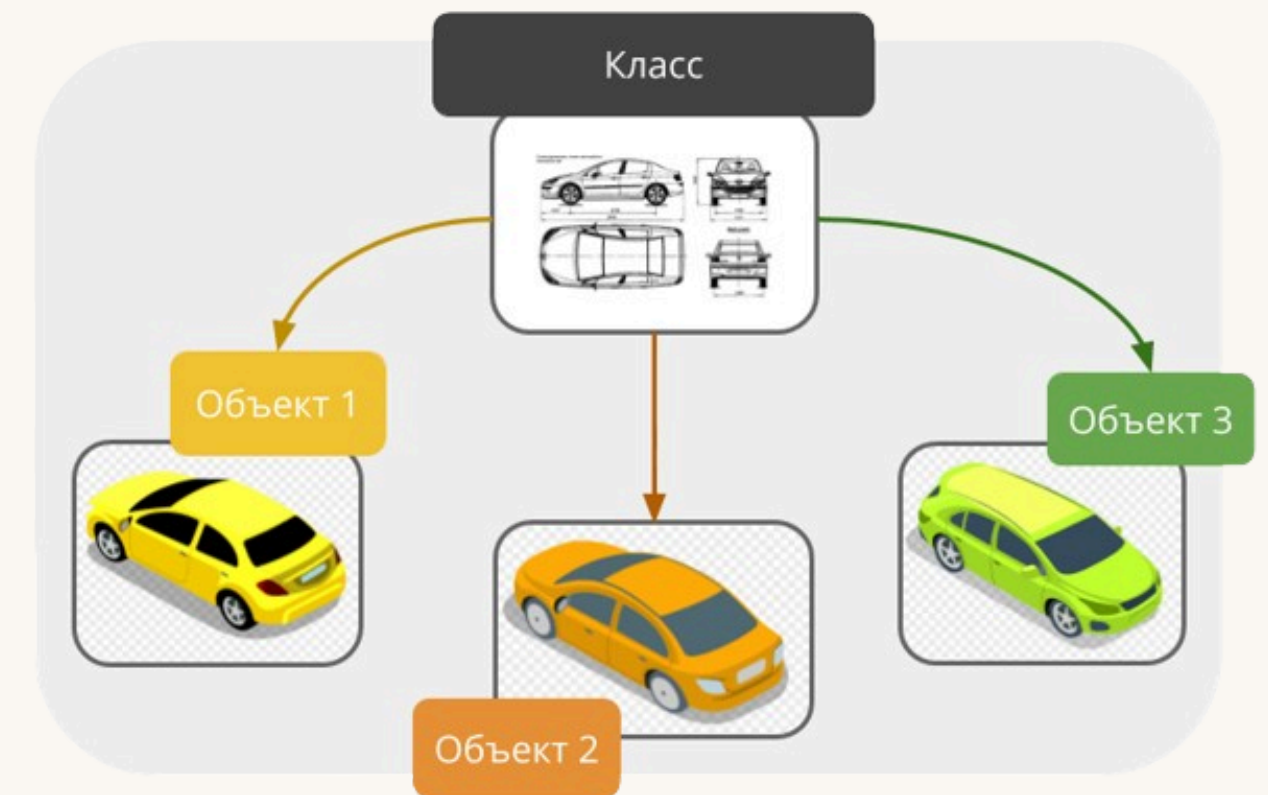
Эти аргументы передаются в метод `__init__`, как в обычную функцию передается обычный аргумент. И внутри тела этого метода они доступны.

Далее, мы напишем:

- **`self.form = form;`**
- **`self.dough = dough;`**

Это означает что именно для этого экземпляра, к которому мы обратились именно его **form** и **dough** будут такими которые мы передали при его инициализации

Объектно Ориентированное Программирование





Что такое ООП.

До этого момента мы не использовали термин Объектно Ориентированное программирование, но теперь мы изучили абстракции, классы и экземпляры и пора познакомится с понятием ООП.

Объектно ориентированное программирование - это подход к программированию, также называемый парадигмой или стилем, когда мы все что можем представляем как классы и их свойства.

То есть по минимуму используем примитивные типы данных (строки, числа, списки) и по максимуму экземпляры классов, которые сами придумали.

Если до этого мы писали просто переменные и функции, которые путешествуют сами по себе по коду программы, теперь мы собираем их в более крупные сущности, собственно в классы, на основе которых и создаются экземпляры.

Дикие переменные и функции

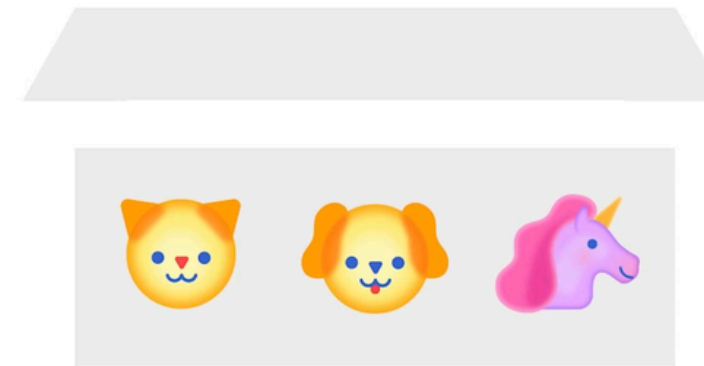


Например раньше мы писали:

- `student_name = ...`
- `student_surname = ...`
- `student_course = ...`

- `get_grade()`

Поля и методы в экземпляре класса



Теперь с использованием классов и их экземпляров это выглядит так:

- `student.name`
- `student.surname`
- `student.course`

- `student.get_grade()`



Что такое ООП.

До этого момента мы не использовали термин Объектно Ориентированное программирование, но теперь мы изучили абстракции, классы и экземпляры и пора познакомится с понятием ООП.

Объектно ориентированное программирование - это подход к программированию, также называемый парадигмой или стилем, когда мы все что можем представляем как классы и их свойства.

То есть по минимуму используем примитивные типы данных (строки, числа, списки) и по максимуму экземпляры классов, которые сами придумали.

Если до этого мы писали просто переменные и функции, которые путешествуют сами по себе по коду программы, теперь мы собираем их в более крупные сущности, собственно в классы, на основе которых и создаются экземпляры.



Применение в практике.

У нас есть уже знакомая нам задача, а именно тест по английскому языку:

```
1 question_1 = "My name __ Vova"
  correct_1 = "is"

  answer = input("Введите ответ")

  if answer == correct_1:
      print("Ответ верный")
  else:
      print("Ответ неверный")
      print(f"Верный {correct_1}")

  >>>

2 class Question:
    def __init__(self, question, answer):
        self.question = question
        self.answer = answer

    def check(self, answer):
        return answer == self.answer

  >>>

3 question_1 = Question("My name __ Vova", "is")
  question_2 = Question("I live __ Moscow", "in")

  questions_list = [question_1, question_2]
```

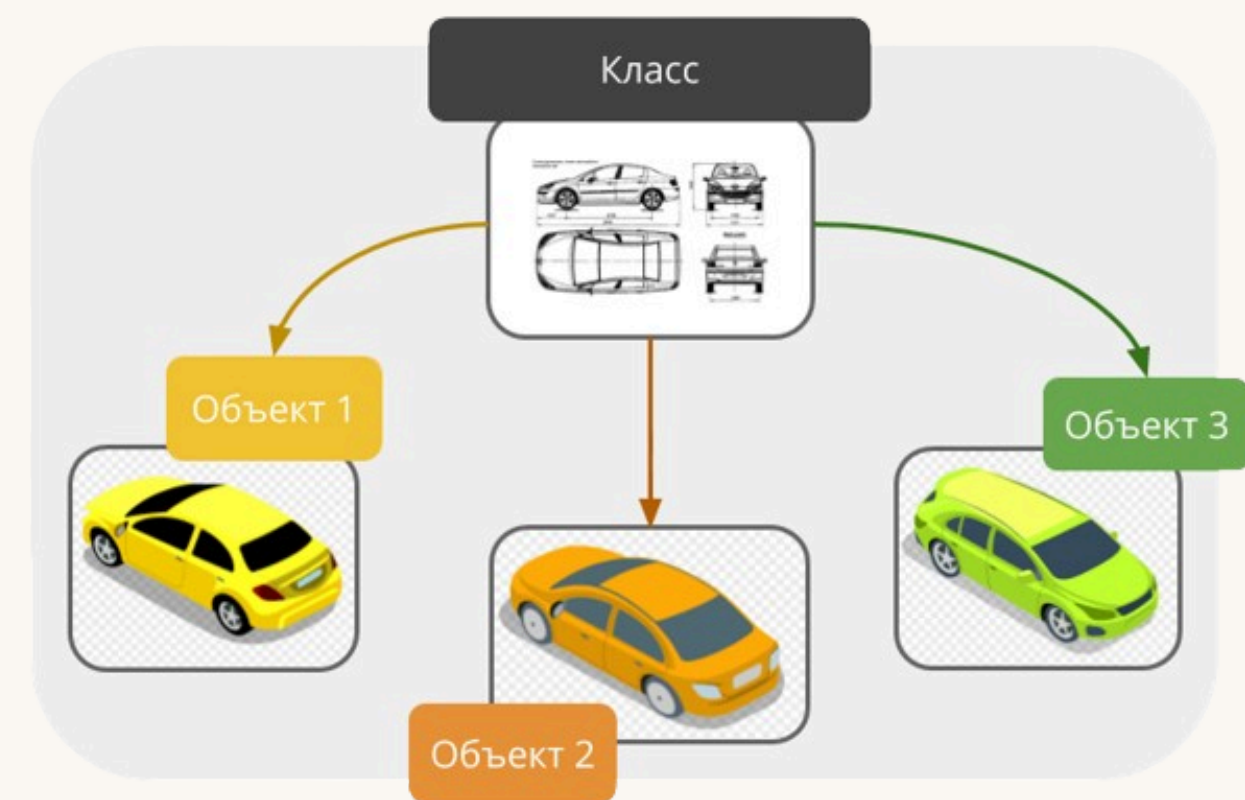
```
4 for question in questions_list:
    print(question.question)
    answer = input("Введите ответ: ")
    if question.check(answer):
        print("Ответ верный")
    else:
        print("Ответ неверный")
        print(f"Верный {question.answer}")

  >>>

5 My name __ Vova
  Введите ответ: is
  Ответ верный
  I live __ Moscow
  Введите ответ: is
  Ответ неверный
  Верный in
```

- 1) Это наше старое приложение по проверке;
- 2) Переписываем его представление под ООП
- 3) Создаем экземпляры, классов помещая их в список;
- 4) Пишем код приложения используя классы их атрибуты и методы;
- 5) Получаем результат.

Фундаментальные принципы ООП





Фундаментальные принципы ООП

Инкапсуляция – механизм сокрытия деталей реализации класса от других объектов. Достигается путем использования модификаторов доступа public, private и protected, которые соответствуют публичным, приватным и защищенным атрибутам.

Наследование – процесс создания нового класса на основе существующего класса. Новый класс, называемый подклассом или производным классом, наследует свойства и методы существующего класса, называемого суперклассом или базовым классом.

Полиморфизм - разное поведение одного и того же метода в разных классах. Например, мы можем сложить два числа, и можем сложить две строки. При этом получим разный результат, так как числа и строки являются разными классами.



Фундаментальные принципы ООП - Инкапсуляция

Рассмотрим эти принципы на примерах и начнем с инкапсуляции, для начала напишем обычный класс:

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book("Дубровский", "Пушкин А.С.") >>>
print(book.title)
print(book.author)
book.title = "Капитанская дочка"
book.author = "А.С. Пушкин"
print(book.title)
print(book.author)
```

Дубровский
Пушкин А.С.
Капитанская дочка
А.С. Пушкин

Был создан класс книга, с атрибутами название (self.title) и автор (self.author). Далее был создан экземпляр класса куда для инициализации были переданы следующие значения атрибутов:

book = Book("Дубровский", "Пушкин А.С.")

Поскольку атрибуты никак не защищены мы можем обратившись к ним изменить их значение, что может быть небезопасно, т.к. пользователь потенциально сможет изменить те или иные значения атрибутов класса



Фундаментальные принципы ООП - Инкапсуляция

Далее сделаем атрибуты title и author класса Book приватными – теперь доступ к ним возможен только внутри класса:

```
class Book:
    def __init__(self, title, author):
        self.__title = title
        self.__author = author

book = Book("Дубровский", "Пушкин А.С.")
print(book.title)
print(book.author)
```

```
>>> AttributeError: 'Book' object has no attribute 'title'
```

Мы сделали атрибуты класса приватными, это значит что мы их скрыли от пользователя пользователя и просто так изменить их значение уже не получится

Но все таки совсем лишать доступа к атрибутам не всегда верно, поэтому мы можем сделать к ним доступ, но доступ при этом будет контролируемым. Для этого мы напишем специальные методы:

```
def get_title(self):
    return self.__title

def set_title(self, title):
    self.__title = title

def get_author(self):
    return self.__author

def set_author(self, author):
    self.__author = author
```

В этом примере методы **get_title()**, **get_author()** являются получающими методами (**геттерами**), которые позволяют нам получать значения приватных атрибутов извне класса. Методы **set_title()**, **set_author()** – устанавливающие методы (**сеттеры**), которые позволяют нам устанавливать значения частных атрибутов извне класса.



Фундаментальные принципы ООП - Инкапсуляция

Вот как теперь выглядит наш класс

```
class Book:
    def __init__(self, title, author):
        self.__title = title
        self.__author = author
```

```
    def get_title(self):
        return self.__title
```

```
    def set_title(self, title):
        self.__title = title
```

```
    def get_author(self):
        return self.__author
```

```
    def set_author(self, author):
        self.__author = author
```

```
book = Book("Дубровский", "Пушкин А.С.")
print(book.get_title())
print(book.get_author())
book.set_title("Капитанская дочка")
book.set_author("А.С. Пушкин")
print(book.get_title())
print(book.get_author())
```

>>>

Дубровский
Пушкин А.С.
Капитанская дочка
А.С. Пушкин

Далее создаем экземпляр класса, и применяем добавленные методы.

Казалось бы зачем что-то дополнительно писать если все можно делать исключительно меняя значение атрибутов.

Создание геттеров и сеттеров позволяет повысить контроль над вашим приложением. Например скрыв какой-то из методов от обычного пользователя. Установить дополнительные правила для сеттеров (чтобы например имя автора или название книги подчинялось каким-то определенным правилам)

Что такое Наследование в ООП.

Для иллюстрации концепции наследования мы определим класс Publication (это будет наш базовый класс), который имеет свойства, общие для всех публикаций – title, author и year, а также общий метод display():

```
class Publication:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year

    def display(self):
        print("Название:", self.title)
        print("Автор:", self.author)
        print("Год выпуска:", self.year)
```

Как я и говорил класс Publication выступает в роли базового класса, что это значит для нас. Это значит что его атрибуты и методы будут доступны для все его классов наследников. То есть нам нет необходимости прописывать данные методы еще раз (если конечно мы не хотим расширить функционал этих методов).

Что такое Наследование в ООП.

Далее создаем классы наследники Book и Magazine и экземпляры этих классов

```
class Book(Publication):
    def __init__(self, title, author, year, code):
        super().__init__(title, author, year)
        self.code = code

    def display(self):
        super().display()
        print(f"Код книги: {self.code}")

class Magazine(Publication):
    def __init__(self, title, author, year, issue_number):
        super().__init__(title, author, year)
        self.issue_number = issue_number

    def display(self):
        super().display()
        print(f"Номер выпуска: {self.issue_number}")

book_1 = Book("Дубровский", "Пушкин А.С.", 2015, "23-025")
magazine_1 = Magazine("Вокруг света", "Коллектив Авторы", 2019, "№05-24")

book_1.display()
print()
magazine_1.display()
```

>>>

Название: Дубровский
Автор: Пушкин А.С.
Год выпуска: 2015
Код книги: 23-025

Название: Вокруг света
Автор: Коллектив Авторы
Год выпуска: 2019
Номер выпуска: №05-24

Мы создали экземпляры класса Book и класса Magazine, мы сможем вызвать метод display() для отображения свойств объекта. Сначала будет вызван метод display() подкласса (Book или Magazine), который в свою очередь вызовет метод display() суперкласса Publication с помощью функции super(). Это позволяет нам повторно использовать код суперкласса и избежать дублирования кода в подклассах:



Фундаментальные принципы ООП Полиморфизм в ООП

Полиморфизм позволяет обращаться с объектами разных классов так, как будто они являются объектами одного класса. Реализовать полиморфизм можно через наследование, интерфейсы и перегрузку методов. Этот подход имеет несколько весомых преимуществ:

- Позволяет использовать различные реализации методов в зависимости от типа объекта, что делает код более универсальным и удобным для использования.
- Уменьшает дублирование кода – можно написать одну функцию для работы с несколькими типами объектов.
- Позволяет использовать общие интерфейсы и абстракции для работы с объектами разных типов.
- Обеспечивает гибкость и расширяемость – можно добавлять новые типы объектов без необходимости изменять существующий код. Это дает возможность разработчикам встраивать новые функции в программу, не нарушая ее существующую функциональность.



Фундаментальные принципы ООП Полиморфизм в ООП

Рассмотрим полиморфизм на примере класса **Confectionary** (кондитерские изделия):

```
class Confectionary:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def describe(self):
        print(f"{self.name} по цене {self.price} руб/кг")

class Cake(Confectionary):
    def describe(self):
        print(f"{self.name} торт стоит {self.price} руб/кг")

class Candy(Confectionary):
    def describe(self):
        print(f"{self.name} конфеты стоимостью {self.price} руб/кг")

class Cookie(Confectionary):
    pass

cake = Cake("Пražский", 1200)
candy = Candy("Шоколадные динозавры", 560)
cookie = Cookie("Овсяное печенье с миндалем", 250)

cake.describe()
candy.describe()
cookie.describe()
```

```
>>> Пражский торт стоит 1200 руб/кг
>>> Шоколадные динозавры конфеты стоимостью 560 руб/кг
>>> Овсяное печенье с миндалем по цене 250 руб/кг
```

В этом примере мы определяем базовый класс **Confectionary**, который имеет атрибуты **name** и **price**, а также метод **describe()**. Затем мы определяем три подкласса класса **Confectionary**: **Cake**, **Candy** и **Cookie**. **Cake** и **Candy** переопределяют метод **describe()** своими собственными реализациями, которые включают тип кондитерского изделия (торт и конфеты соответственно), а **Cookie** наследует дефолтный метод **describe()** от **Confectionary**.

Если создать **экземпляры этих классов и вызвать их методы describe()**, можно убедиться, что *результат зависит от реализации метода в каждом конкретном подклассе (в данном примере это и есть полиморфизм)*.

Фундаментальные принципы ООП все вместе

Рассмотрим пример где используются все 3 изученных нами принципа ООП:

Шаг 1: сначала создаем базовый класс:

```
class Beverage:
    def __init__(self, name, size, price):
        self._name = name
        self._size = size
        self._price = price

    def get_name(self):
        return self._name

    def get_size(self):
        return self._size

    def get_price(self):
        return self._price

    def set_price(self, price):
        self._price = price

    def describe(self):
        return f'{self._size} л газировки "{self._name}" стоит {self._price} руб.'
```

Теперь разберем на примере класса Beverage (напиток) взаимодействие полиморфизма с другими концепциями ООП. Beverage – родительский класс, который содержит:

- атрибуты названия, объема и цены;
- методы для получения и установки этих атрибутов;
- метод для вывода описания напитка.

Фундаментальные принципы ООП все вместе

Рассмотрим пример где используются все 3 изученных нами принципа ООП:

Шаг 2: далее создаем классы наследники:

```
class Soda(Beverage):
    def __init__(self, name, size, price, flavor):
        super().__init__(name, size, price)
        self._flavor = flavor

    def get_flavor(self):
        return self._flavor

    def describe(self):
        return f'{self._size} л газировки "{self._name}" со вкусом "{self._flavor}" стоит {self._price} руб.'

class DietSoda(Soda):
    def __init__(self, name, size, price, flavor):
        super().__init__(name, size, price, flavor)

    def describe(self):
        return f'{self._size} л диетической газировки "{self._name}" со вкусом "{self._flavor}" стоит {self._price} руб.'
```

Soda (газировка) – дочерний класс Beverage, у него есть дополнительный атрибут **flavor** (вкус) и собственный метод **describe()**, включающий flavor. **DietSoda** – еще один дочерний класс Soda, который наследует все атрибуты и методы Soda, но переопределяет метод **describe()**, чтобы указать, что газировка является диетической:

Фундаментальные принципы ООП все вместе

Рассмотрим пример где используются все 3 изученных нами принципа ООП:

Шаг 3: создаем экземпляры каждого класса и применяем к ним доступные для них методы:

```
beverage = Beverage("Буратино", 1.5, 150)
print(beverage.describe())
beverage.set_price(170)
print(beverage.describe())
print()
regular_soda = Soda('Sprite', 0.33, 45, 'лимон')
print(regular_soda.describe())
regular_soda.set_price(55)
print(regular_soda.describe())
print()
diet_soda = DietSoda('Cola Zero', 0.5, 70, 'вкус колы')
print(diet_soda.describe())
diet_soda.set_price(85)
print(diet_soda.describe())
```

>>> 1.5 л газировки "Буратино" стоит 150 руб.
1.5 л газировки "Буратино" стоит 170 руб.

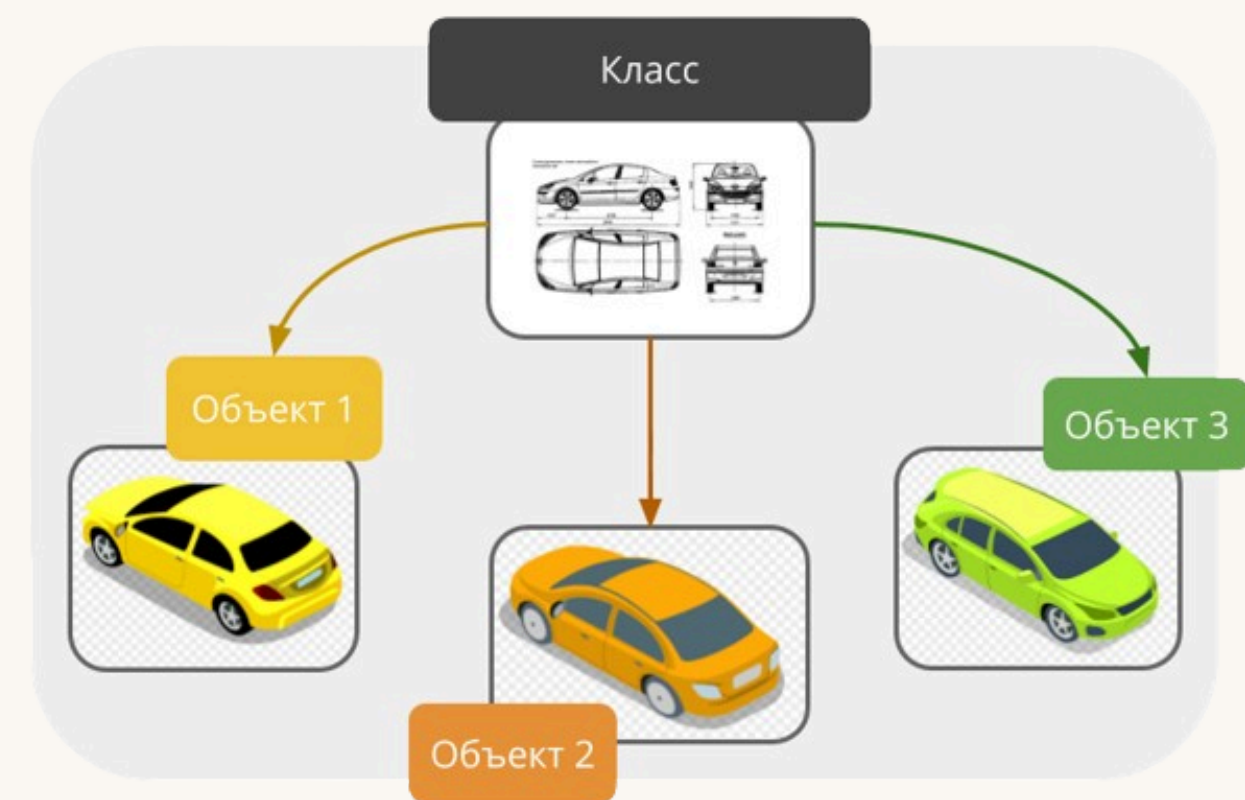
0.33 л газировки "Sprite" со вкусом "лимон" стоит 45 руб.
0.33 л газировки "Sprite" со вкусом "лимон" стоит 55 руб.

0.5 л диетической газировки "Cola Zero" со вкусом "вкус колы" стоит 70 руб.
0.5 л диетической газировки "Cola Zero" со вкусом "вкус колы" стоит 85 руб.

Этот пример демонстрирует:

- **Инкапсуляцию**, поскольку атрибуты защищены символами подчеркивания и могут быть доступны только через методы **getter** и **setter**.
- **Наследование**, поскольку **Soda** и **DietSoda** наследуют атрибуты и метод от **Beverage**.
- **Полиморфизм**, поскольку каждый класс имеет свою собственную версию метода **describe()**, который возвращает различные результаты в зависимости от конкретного класса

Переменные класса



__dict__

При создании объекты Python наделяются небольшим набором заранее определенных свойств и методов. Они есть у каждого объекта, хотите вы того или нет. Одним из таких свойств является переменная с именем **__dict__** (сокращенно от «dictionary», словарь).

Эта переменная содержит имена и значения всех свойств (переменных), которые в данный момент есть у объекта. Мы будем ее использовать для безопасного представления содержимого объекта.

```
class ExampleClass:
    def __init__(self, first=1, second=2):
        self.first = first
        self.second = second

    def set_second(self, second):
        self.second = second
```

```
example_object_1 = ExampleClass()
example_object_2 = ExampleClass(2)
example_object_2.set_second(3)
example_object_3 = ExampleClass(4)
example_object_3.third = 5
print(example_object_1.__dict__)
print(example_object_2.__dict__)
print(example_object_3.__dict__)
```

```
>>> {'first': 1, 'second': 2}
{'first': 2, 'second': 3}
{'first': 4, 'second': 2, 'third': 5}
```

У класса под названием «ExampleClass» есть конструктор, который в обязательном порядке создает переменные экземпляра с именами «first» и «second» и устанавливает их значение, которое передается через первый и второй аргументы (с точки зрения пользователя класса) или второй аргумент (с точки зрения конструктора).

У класса также есть метод, который может изменять значение второй переменной

Далее мы создаем 3 объекта класса:

- у объекта `example_object_1` есть только два атрибута со значениями по умолчанию
- у объекта `example_object_2` есть уже значение первого атрибута: 2, второй мы меняем при помощи сеттера: было 2 стало 3
- к объекту `example_object_3` мы на ходу добавили атрибут `third`, хотя мы это сделали вне кода класса, это возможно и вполне допустимо.

Из результата следует изменение переменной экземпляра объекта не влияет на все остальные объекты. Переменные экземпляра идеально изолированы друг от друга.



Переменные класса

Переменная класса — это свойство, которое существует только в единственном экземпляре и хранится вне объекта.

Примечание: переменной экземпляра не существует, если в классе нет объекта. А переменная класса существует в единственном экземпляре, даже если в классе нет объектов.

Создание переменных класса отличается от создания экземпляров. Пример ниже даст вам больше информации:

```
class ExampleClass:
    counter = 0

    def __init__(self, first=1):
        self.__first = first
        ExampleClass.counter += 1

example_object_1 = ExampleClass()
example_object_2 = ExampleClass(2)
example_object_3 = ExampleClass(4)
print(example_object_1.__dict__, example_object_1.counter)
print(example_object_2.__dict__, example_object_2.counter)
print(example_object_3.__dict__, example_object_3.counter)
```

```
>>> {'_ExampleClass__first': 1} 3
{'_ExampleClass__first': 2} 3
{'_ExampleClass__first': 4} 3
```

При определении класса есть присваивание — оно присваивает переменной counter значение 0; инициализация переменной внутри класса, но вне метода этого класса, превращает переменную в переменную класса;

Доступ к такой переменной выглядит так же, как доступ к любому атрибуту экземпляра — он находится в теле конструктора; как видите, конструктор увеличивает переменную на единицу; по сути, переменная считает все созданные объекты.



Переменные класса

Какие выводы мы можем сделать - из примера на предыдущей странице:

- переменные класса не отображаются в объекте `__dict__` (это нормально, поскольку переменные класса не являются частями объекта), но всегда можно попытаться найти переменную с тем же именем, но на уровне класса;
- переменная класса всегда представляет одно и то же значение во всех экземплярах (объектах) класса (в том случае если сначала мы создаем объекты и только потом уже обращаемся к переменной).



Переменные класса

Изменение статуса переменной класса приведет к уже знакомому нам эффекту (когда мы ее хотим скрыть)

```
class ExampleClass:
    __counter = 0

    def __init__(self, val=1):
        self.__first = val
        ExampleClass.__counter += 1
```

```
exampleObject1 = ExampleClass()
exampleObject2 = ExampleClass(2)
exampleObject3 = ExampleClass(4)
print(exampleObject1.__dict__, exampleObject1._ExampleClass__counter)
print(exampleObject2.__dict__, exampleObject2._ExampleClass__counter)
print(exampleObject3.__dict__, exampleObject3._ExampleClass__counter)
```

```
>>> {'_ExampleClass__first': 1} 3
{'_ExampleClass__first': 2} 3
{'_ExampleClass__first': 4} 3
```

Результат выполнения кода такой же как и в предыдущем примере, но обратите внимание как мы обращаемся к переменной класса.



Переменные класса

Я уже упоминал, что переменные класса существуют, даже если экземпляр (объект) класса не был создан. Воспользуемся этой возможностью, чтобы показать вам, разницу между двумя переменными `__dict__`, той, что из класса и той, что из объекта. Посмотрите на код.

```
class ExampleClass:
    varia = 1

    def __init__(self, val):
        ExampleClass.varia = val

print(ExampleClass.__dict__)
exampleObject = ExampleClass(2)

print(ExampleClass.__dict__)
print(exampleObject.__dict__)
```

V V V

```
{'__module__': '__main__', 'varia': 1, '__init__': <function ExampleClass.__init__ at 0x000001DA241DB600>,
'__dict__': <attribute '__dict__' of 'ExampleClass' objects>,
'__weakref__': <attribute '__weakref__' of 'ExampleClass' objects>, '__doc__': None}
{'__module__': '__main__', 'varia': 2, '__init__': <function ExampleClass.__init__ at 0x000001DA241DB600>,
'__dict__': <attribute '__dict__' of 'ExampleClass' objects>,
'__weakref__': <attribute '__weakref__' of 'ExampleClass' objects>, '__doc__': None}
{}
```

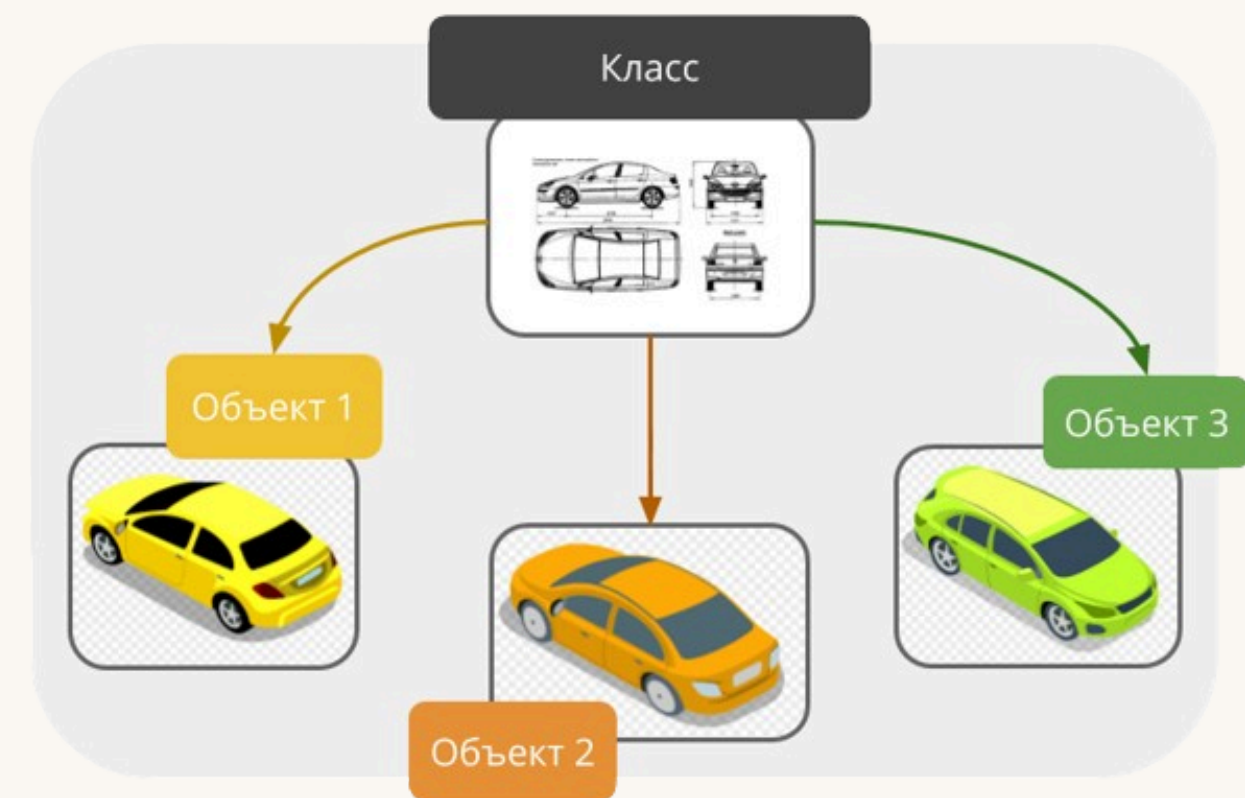
**В консоли это все выводится в 3 строки*

Проанализируем его внимательно:

- в коде определяется один класс с именем `ExampleClass`;
- класс определяет одну переменную класса с именем `varia`;
- конструктор класса устанавливает переменную со значением параметра;
- именованная переменная — самая важная часть этого примера, потому что:
 - если изменить присваивание на `self.varia = val`, то будет создана переменная экземпляра с тем же именем, что и у класса;
 - если изменить присваивание на `varia = val`, то это повлияет на локальную переменную метода (я настоятельно рекомендую вам протестировать обе вышеупомянутые ситуации — это поможет вам запомнить разницу);
- первая строка кода, который вне класса, выведет на экран значение атрибута `ExampleClass.varia` (значение используется до того, как будет создан самый первый объект класса).

Результат запуска будет вот таким. класс `__dict__` содержит гораздо больше данных, чем одноименный объект. Большая часть этих данных пока не нужна, а та, на которую мы нужно обратить внимание, показывает текущее значение `varia`.

Проверка существования атрибута





Набор атрибутов объекта

Инстанциация (порядок создания) объектов в Python поднимает одну важную проблему — в отличие от других языков программирования, в Python не все объекты одного класса имеют одинаковый набор свойств.

Вот как в этом примере. Внимательно посмотрите на него.

```
class ExampleClass:
    def __init__(self, val):
        if val % 2 != 0:
            self.a = 1
        else:
            self.b = 1

exampleObject = ExampleClass(1)
print(exampleObject.a)
print(exampleObject.b)

>>> print(exampleObject.b)
      ^^^^^^^^^^^^^^^^^^^
AttributeError: 'ExampleClass' object has no attribute 'b'
```

Объект, созданный конструктором, может иметь только один из двух возможных атрибутов: **a** или **b**.
Выполнение кода даст:

- **1** для **a**;
- ошибку: **AttributeError: 'ExampleClass' object has no attribute 'b'** для **b**



Набор атрибутов объекта: как отмахнуться от проблемы

Инструкция try-except позволяет избежать проблем с несуществующими свойствами.

В этом нет ничего сложного:

```
class ExampleClass:
    def __init__(self, val):
        if val % 2 != 0:
            self.a = 1
        else:
            self.b = 1

exampleObject = ExampleClass(1)

try:
    print(exampleObject.a)
except AttributeError:
    print("Атрибут exampleObject.a не существует")

try:
    print(exampleObject.b)
except AttributeError:
    print("Атрибут exampleObject.b не существует")
```

>>>

1

Атрибут exampleObject.b не существует

Как видите, это не очень сложно. Но, на самом деле, мы только что просто отмахнулись от проблемы. К счастью, есть один действенный способ.



Набор атрибутов объекта: как правильно решить проблему

У Python есть функция, которая может безопасно проверять, содержит ли какой-либо объект/класс указанное свойство. Функция называется «hasattr» и принимает два аргумента:

- проверяемый класс или объект;
- имя атрибута, о существовании которого она сообщит (примечание: это должна быть строка с именем атрибута, а не просто имя)

Функция возвращает True или False. Ее можно использовать следующим образом:

```
class ExampleClass:
    def __init__(self, val):

        if val % 2 != 0:
            self.a = 1
        else:
            self.b = 1

exampleObject = ExampleClass(1)
if hasattr(exampleObject, 'a'):
    print(exampleObject.a)
if hasattr(exampleObject, 'b'):
    print(exampleObject.b)
```

>>> 1

Как видите все предельно просто:

- если атрибут есть (**hasattr() is True**) мы его выводим в консоль;
- если атрибута нет (**hasattr() is False**) то ничего и не делаем.



hasattr() для атрибутов класса

Функция `hasattr()` работает и с классами. Ее точно так же можно использовать, чтобы узнать, доступен ли атрибут класса.

```
class ExampleClass:
    attr = 1

>>> True
False

print(hasattr(ExampleClass, 'attr'))
print(hasattr(ExampleClass, 'prop'))
```

Функция возвращает `True`, если указанный класс содержит данный атрибут, в противном случае `False`. Что и демонстрирует нам запуск данного кода.

Разберем еще 1 пример но уже добавим экземпляры:

```
class ExampleClass:
    a = 1

    def __init__(self):
        self.b = 2

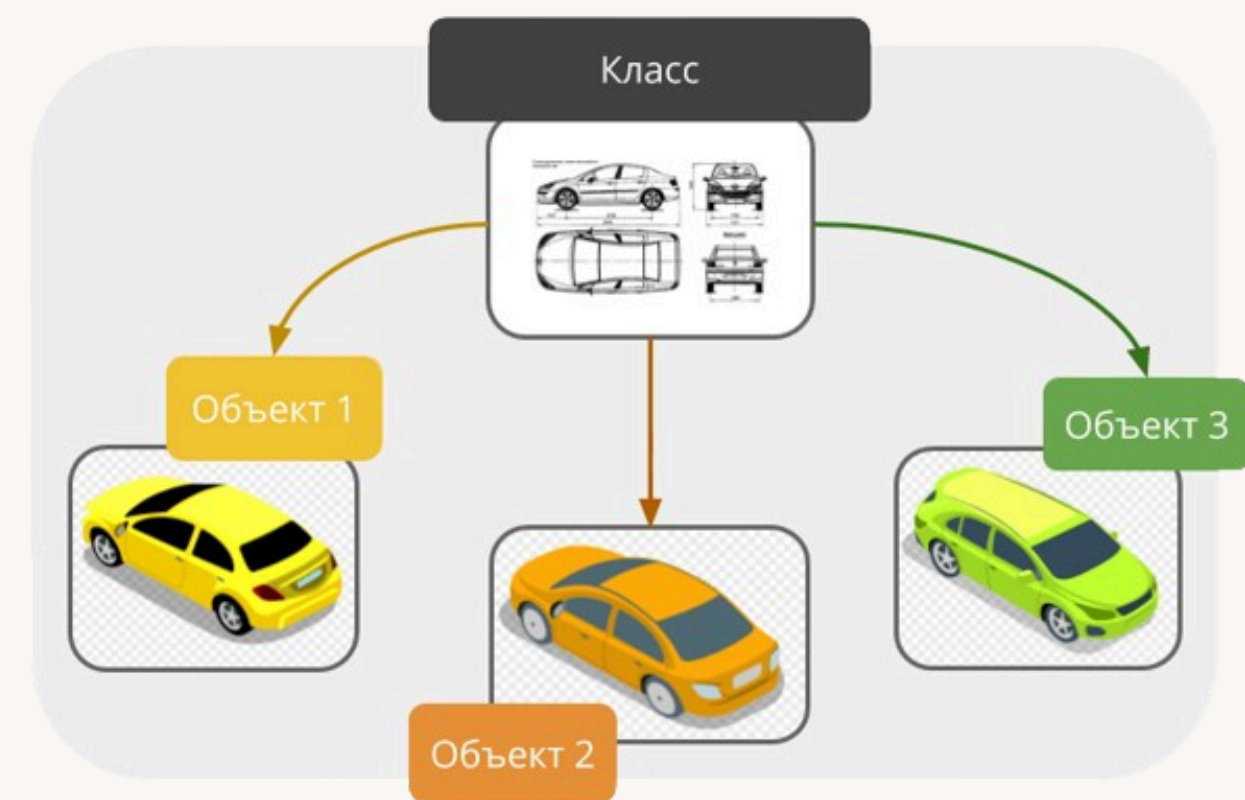
>>> True
True
True
False

exampleObject = ExampleClass()
print(hasattr(exampleObject, 'a'))
print(hasattr(exampleObject, 'b'))
print(hasattr(ExampleClass, 'a'))
print(hasattr(ExampleClass, 'b'))
```

Поскольку объект класса обладает как своими атрибутами так и атрибутами класса мы получаем **True в обоих случаях**.

А вот сам класс не может пользоваться атрибутами объекта поэтому и получаем **False** для 'b'

Внутренняя жизнь классов и объектов





Что нам показывает `__dict__`

У любого класса и объекта в Python есть набор полезных атрибутов (я уже упоминал что так зачастую одним словом называют и поля, и методы), благодаря которым можно узнать, что умеет этот класс или объект.

С одним из таких мы уже познакомились это дандер метод `__dict__`.

Давайте посмотрим, как это работает с методами:

```
class Classy:
    varia = 1

    def __init__(self):
        self.var = 2

    def method(self):
        pass

    def __hidden(self):
        pass

obj = Classy()
print(obj.__dict__)
print(Classy.__dict__)
```

`>>>`

```
{'var': 2}
{'__module__': '__main__', 'varia': 1, '__init__': <function Classy.__init__ at 0x000001DE3BA9B600>,
'method': <function Classy.method at 0x000001DE3BA9B6A0>,
'_Classy__hidden': <function Classy.__hidden at 0x000001DE3BA9B740>,
'__dict__': <attribute '__dict__' of 'Classy' objects>,
'__weakref__': <attribute '__weakref__' of 'Classy' objects>, '__doc__': None}
```

- экземпляра содержит только его `self.var`;
- Сам класс как видим внутри содержит намного больше:
- `'__module__': '__main__'` - расположение в каком модуле он находится;
 - `'varia': 1` - атрибут класса;
 - `'__init__': <function Classy.__init__ at 0x000001DE3BA9B600>` - метод (конструктор) `'__init__'` и его адрес в памяти;
 - `'method': <function Classy.method at 0x000001DE3BA9B6A0>` - метод `'method'` и его адрес в памяти;
 - `'_Classy__hidden': <function Classy.__hidden at 0x000001DE3BA9B740>` - скрытый метод `'_Classy__hidden'` и его адрес в памяти;
 - `'__dict__': <attribute '__dict__' of 'Classy' objects>` - дандер метод `'__dict__'` - атрибут (метод) объектов класса `'Classy'`, ну и конечно он есть у самого класса;
 - `'__weakref__': <attribute '__weakref__' of 'Classy' objects>` - слабая ссылка на объект ("Слабой" ссылки не достаточно, чтобы объект оставался "живым": когда на объект ссылаются только "слабые" ссылки, сборщик мусора удаляет объект и использует память для других объектов. Однако, пока объект не удалён, "слабая" ссылка может вернуть объект, даже если не осталось обычных ссылок на объект. Используется например для реализации кэшей <https://habr.com/ru/articles/599411/>);
 - `'__doc__': None` - метод `'__doc__'` для извлечения документации из класса (но тут нет докстрингов поэтому `None`)



Что нам показывает `__name__` и `__module__`

`__dict__` — это словарь. Стоит упомянуть еще одно встроенное свойство — `__name__`, которое является строкой. Этот метод содержит название класса. Ничего необычного, это просто строка. Обратите внимание, что атрибута `__name__` нет в объекте, он существует только внутри классов.

Если надо найти класс конкретного объекта, можно использовать функцию с именем `type()`, которая (среди прочего) также может найти класс, из которого был создан экземпляр объекта.

```
class Classy:
    pass

>>> Classy
<class '__main__.Classy'>
Classy

print(Classy.__name__)
obj = Classy()
print(type(obj))
print(type(obj).__name__)
```

Получилось немного масло масляное :)

- `print(Classy.__name__)` - покажи имя Classy и оно Classy;
- `print(type(obj))` - тип объекта, `<class '__main__.Classy'>` : класс и модуль где он располагается
- `print(type(obj).__name__)` -покажи имя класса Classy расположенном (в данном случае) в `__main__`

**строка кода: `print(obj.__name__)` вызовет ошибку.*

`__module__` — тоже строка. Она хранит имя модуля, который содержит определение класса.

```
class Classy:

    def __init__(self, name):
        self.name = name

>>> __main__
__main__

if __name__ == "__main__":
    print(Classy.__module__)
    obj = Classy("ClassyObj")
    print(obj.__module__)
```

```
from _06_inner_life_ import Classy

>>> _06_inner_life_

obj = Classy("ClassyObj")
print(obj.__module__)
```

Как вы уже знаете любой модуль с именем `__main__` на самом деле не модуль, а файл, запущенный в данный момент. И как видим при импорте все равно модуль показывает именно где находится а не где запускается.



Что нам показывает `__bases__`

`__bases__` — это кортеж. Кортеж содержит классы (а не имена классов), которые являются прямыми суперклассами для класса. Порядок такой же, как и в определении класса.

Рассмотрим на простом примере, более того я покажу как использовать этот метод, когда будем обсуждать объективные аспекты исключений.

Примечание: только у классов есть этот атрибут, у объектов его нет.

```
class SuperOne:
    pass

class SuperMixin:
    pass

class Sub(SuperMixin, SuperOne):
    pass

>>> ( object )
>>> ( object )
>>> ( SuperMixin SuperOne )

def print_bases(cls):
    print('(' , end='')
    for x in cls.__bases__:
        print(x.__name__, end=' ')
    print(')')

print_bases(SuperOne)
print_bases(SuperMixin)
print_bases(Sub)
```

- класс без явных суперклассов указывает на объект (заранее определенный класс Python) как на своего прямого предка.
- класс наследник (subclass) указывает на классов родителей, они как раз указаны явно.

Рефлексия и интроспекция



Рефлексия и интроспекция что это такое?

Эти средства позволяют Python-программисту выполнять два важных действия, которые характерны для многих объектных языков. А именно:

- **интроспекция** — способность программы проверять тип или свойства объекта во время выполнения;
- **рефлексия (отражение)** — способность программы манипулировать значениями, свойствами и/или функциями объекта во время выполнения.

Другими словами, вам не нужно знать полное определение класса/объекта, чтобы манипулировать объектом, поскольку объект и/или его класс содержат метаданные, позволяющие распознавать его функции во время выполнения программы.

introspection

the ability of a program to examine the type or properties of an object at runtime

reflection

the ability of a program to manipulate the values, properties and/or functions of an object at runtime



Анализ классов

Что Python позволяет узнать о классах? Ответ прост — все.

Благодаря рефлексии и интроспекции программист может делать что угодно с любым объектом.

Проанализируем следующий код:

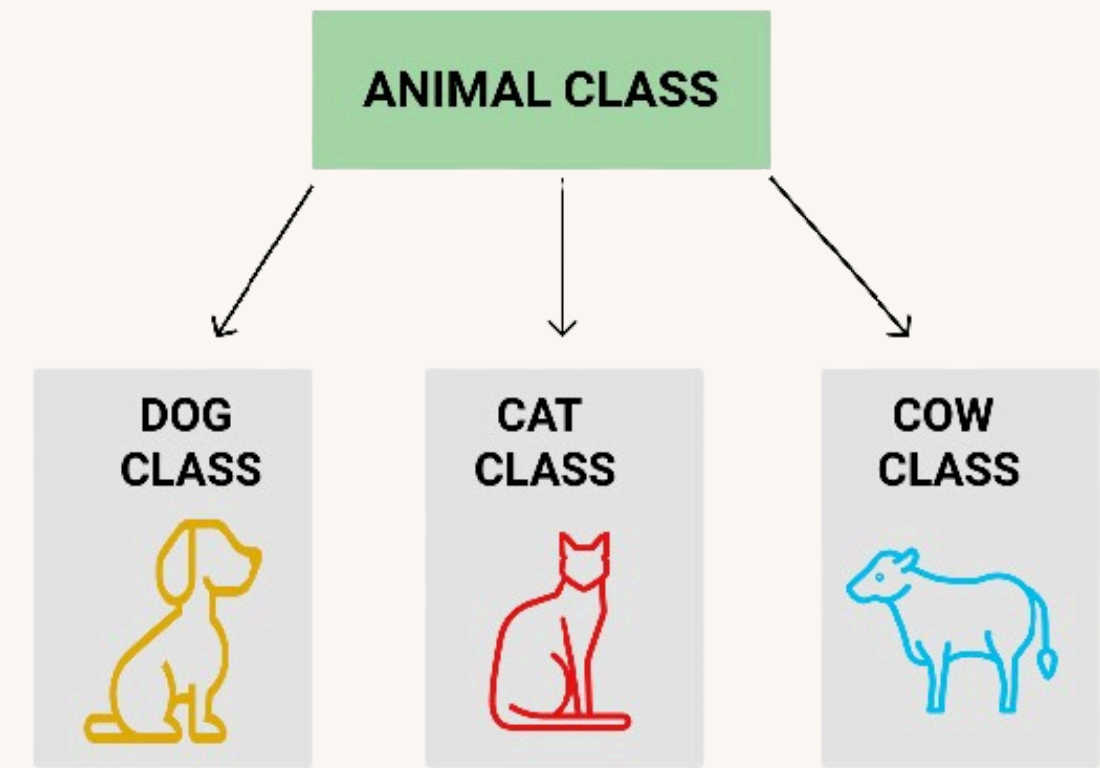
```
1 class MyClass:
2     pass
3
4
5 obj = MyClass()
6 obj.a = 1
7 obj.b = 2
8 obj.i = 3
9 obj.ireal = 3.5
10 obj.integer = 4
11 obj.z = 5
12
13
14 def inc_ints_i(obj):
15     for name in obj.__dict__.keys():
16         if name.startswith('i'):
17             val = getattr(obj, name)
18             if isinstance(val, int):
19                 setattr(obj, name, val + 1)
20
21
22 print(obj.__dict__)
23 inc_ints_i(obj)
24 print(obj.__dict__)
```

```
>>> {'a': 1, 'b': 2, 'i': 3, 'ireal': 3.5, 'integer': 4, 'z': 5}
{'a': 1, 'b': 2, 'i': 4, 'ireal': 3.5, 'integer': 5, 'z': 5}
```

Функция с именем **inc_ints_i()** получает объект любого класса, просматривает его содержимое, чтобы найти все целочисленные атрибуты с именами, которые начинаются на **i**, и увеличивает их значение на единицу. Вот как это работает:

- строка 1: определить очень простой класс...
- строки с 6 по 10 ...и заполнить его атрибутами;
- строка 14: это наша функция!
- строка 15: запускаем цикл по ключам `'__dict__'` объекта `'obj'`...
- строка 16: ...если имя атрибута начинается на `'i'`....
- строка 17: ... использовать функцию `getattr()`, чтобы получить ее текущее значение; примечание: `getattr()` принимает два аргумента: объект и имя его свойства (в виде строки) и возвращает значение текущего атрибута;
- строка 18: проверить, является ли значение `integer`, и использовать для этого функцию `isinstance()` (мы обсудим это позже);
- строка 19: если проверка прошла успешно, увеличить значение атрибута, используя функцию `setattr()`; функция принимает три аргумента: объект, имя свойства (в виде строки) и новое значение свойства.

Проверка наследования (Inheritance)



issubclass для чего используется?

У Python есть функция, которая распознает отношения между двумя классами и, хотя ее принцип работы несложен, она способна проверить, является ли определенный класс подклассом другого класса. Вот как это выглядит:

- **issubclass(ClassOne, ClassTwo)**

Функция возвращает True (Правда), если ClassOne является подклассом ClassTwo, в противном случае — False (Ложь).

```
class Vehicle:
    pass
```

```
class LandVehicle(Vehicle):
    pass
```

```
class TrackedVehicle(LandVehicle):
    pass
```

```
for cls1 in [Vehicle, LandVehicle, TrackedVehicle]:
    for cls2 in [Vehicle, LandVehicle, TrackedVehicle]:
        print(issubclass(cls1, cls2), end="\t")
    print()
```

```
>>> True    False  False
      True    True   False
      True    True   True
```

Посмотрим, как это работает. В коде есть две вложенные петли. Их цель — проверить все возможные упорядоченные пары классов и вывести результат проверки на отношение подкласс-суперкласс в каждой паре.

Упорядочим, результат:

↓ является подклассом для →	Vehicle	LandVehicle	TrackedVehicle
Vehicle	True	False	False
LandVehicle	True	True	False
TrackedVehicle	True	True	True

Необходимо сделать одно важное замечание: каждый класс считается собственным подклассом.



isinstance для чего используется?

Как вы уже знаете, объект является воплощением класса. То есть объект можно сравнить с тортом, который испекли по рецепту, который находится внутри класса. Из-за этого у нас могут быть некоторые проблемы.

Предположим, у вас есть торт (например, в виде аргумента, переданного в вашу функцию). Вы хотите знать, по какому рецепту его приготовили. Почему? Потому что вы должны понимать, чего от него ожидать, например, есть ли в нем орехи, а для некоторых людей это критически важная информация. Точно так же наличие (или отсутствие) у объекта определенных характеристик может иметь решающее значение. Другими словами, принадлежит ли этот объект определенному классу или нет.

Это можно узнать с помощью функции **isinstance()**:

- **isinstance(objectName, ClassName)**

Функция возвращает True, если объект является экземпляром класса, в противном случае — False. Что такое по сути экземпляр класса? Это означает, что объект (торт) был приготовлен по рецепту, который содержится либо в классе, либо в одном из его суперклассов.

Примечание: если у подкласса такой же набор свойств, как и у любого из его суперклассов, это означает, что объекты подкласса могут делать то же, что и объекты, которые произошли от этого суперкласса, следовательно, это экземпляр его домашнего класса и любого из его суперклассов.



isinstance пример:

Проверим это на примере, создав три объекта, по одному для каждого из классов. Далее, используем два вложенных цикла, проверяем все возможные пары объект-класс, чтобы выяснить, являются ли объекты экземплярами классов.

```
class Vehicle:
    pass
```

```
class LandVehicle(Vehicle):
    pass
```

```
class TrackedVehicle(LandVehicle):
    pass
```

```
myVehicle = Vehicle()
```

```
myLandVehicle = LandVehicle()
```

```
myTrackedVehicle = TrackedVehicle()
```

```
for obj in [myVehicle, myLandVehicle, myTrackedVehicle]:
    for cls in [Vehicle, LandVehicle, TrackedVehicle]:
        print(isinstance(obj, cls), end="\t")
    print()
```

```
>>> True    False  False
      True    True   False
      True    True   True
```

Упорядочим результаты:

↓ является экземпляром для →	Vehicle	LandVehicle	TrackedVehicle
myVehicle	True	False	False
myLandVehicle	True	True	False
myTrackedVehicle	True	True	True

Цикл берет сначала объект того или иного класса после чего проверяет принадлежит ли объект к этому классу и если **объект myVehicle относится только к классу Vehicle**, то объекты **myLandVehicle** и **myTrackedVehicle** относятся как к своему непосредственному классу так и к классу-родителю (суперклассу)



Наследование: оператор is

Также стоит упомянуть еще один оператор Python, так как он ссылается непосредственно на объекты:

- **objectOne is objectTwo**

Оператор `is` проверяет, относятся ли две переменные (тут, `objectOne` и `objectTwo`) к одному и тому же объекту.

Не забывайте, что переменные хранят не сами объекты, а только дескрипторы, указывающие на внутреннюю память Python.

Если присвоить значение переменной объекта другой переменной, то Python скопирует не объект, а только его дескриптор. Вот почему в определенных ситуациях операторы вроде `is` могут быть очень полезными.

```
class SampleClass:
    def __init__(self, val):
        self.val = val

ob1 = SampleClass(0)
ob2 = SampleClass(2)
ob3 = ob1
ob3.val += 1
print(ob1 is ob2)
print(ob2 is ob3)
print(ob3 is ob1)
print(ob1.val, ob2.val, ob3.val)
str1 = "Mary had a little "
str2 = "Mary had a little lamb"
str1 += "lamb"
print(str1 == str2, str1 is str2)
```

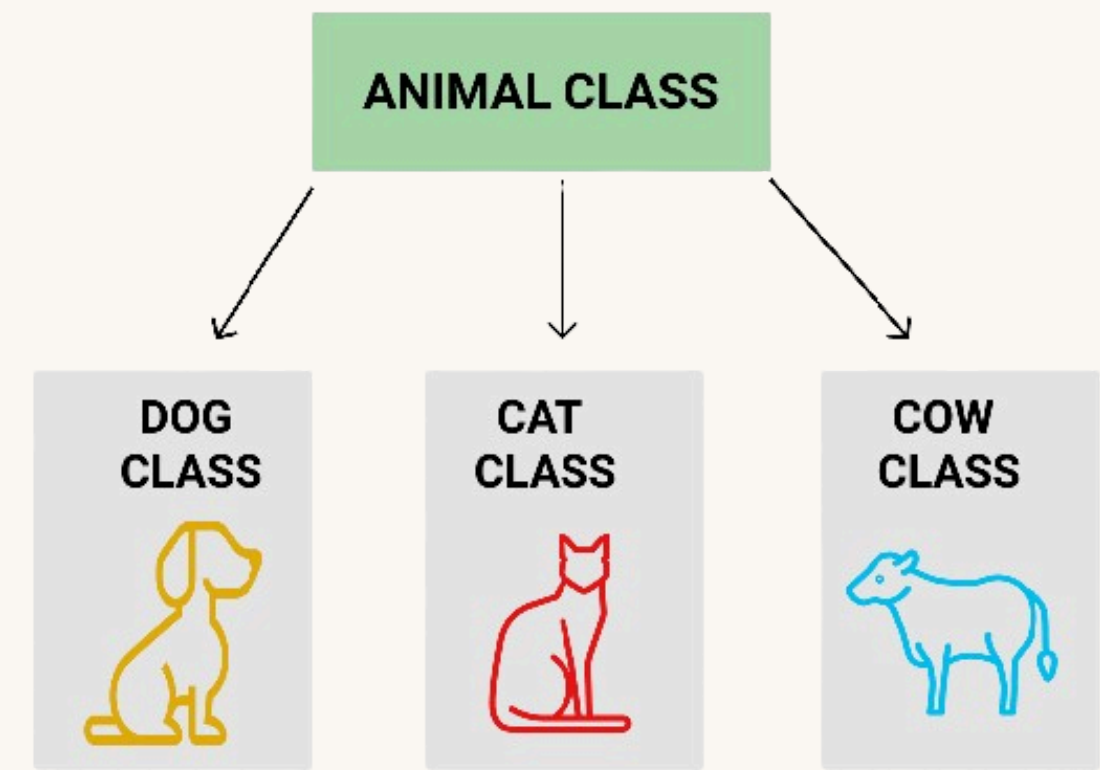
>>> False
False
True
1 2 1
True False

Проанализируем код:

- в коде есть очень простой класс и простой конструктор, который создает только одно свойство. Класс создает два объекта. Первый из них затем присваивается другой переменной, и его свойство `val` увеличивается на единицу.
- затем оператор `is` применяется трижды, чтобы проверить все возможные пары объектов, и выводятся все значения свойств `val`.
- в последней части кода проводится еще один эксперимент. После трех присваиваний обе строки содержат одинаковые тексты, но эти тексты хранятся в разных объектах.

Результаты доказывают, что **ob1** и **ob3** — на самом деле один и тот же объект, а **str1** и **str2** — нет, несмотря на то, что их содержимое одинаково

Построение иерархии классов





Построение иерархии классов

Построение иерархии классов — это не просто искусство ради искусства.

Можно поделить одну задачу между классами и решить, какой из них должен быть расположен вверху, а какой — внизу иерархии, а для этого необходимо тщательно проанализировать эту задачу. Но прежде чем мы покажем вам, как это делается (и как этого не надо делать), нужно рассмотреть один интересный эффект. В нем нет ничего необычного (это просто следствие общих правил наследования), но он может помочь вам понять, как работают некоторые коды, и как этот эффект можно использовать для создания гибкого набора классов. Посмотрим на этот код.

```
class One:
    def do_it(self):
        print("doit from One")

    def do_anything(self):
        self.do_it()

class Two(One):
    def do_it(self):
        print("doit from Two")

one = One()
two = Two()
one.do_anything()
two.do_anything()
```

>>> doit from One
doit from Two

Проанализируем код:

- в нем два класса, которые называются One и Two, класс Two происходит от класса One. Ничего особенного. Тем не менее, метод do_it() выглядит необычно.
- метод do_it() определяется дважды: сначала внутри метода One, а затем внутри метода Two. Суть примера заключается в том, что он вызывается только один раз — внутри One.

Вопрос в том, какой из этих двух методов будет вызываться в двух последних строках кода.

- Первый вызов кажется простым, и это на самом деле так — вызов doanything() из объекта, который называется one, активирует первый из методов.
- Второй вызов требует некоторого внимания. Он тоже простой, если вы помните, как Python находит компоненты класса. Второй вызов запустит doit(), который находится внутри класса Two, несмотря на то, что вызов происходит в пределах класса One.



Построение иерархии классов

Метод, переопределенный в любом из подклассов и изменяющий таким образом поведение суперкласса, называется **виртуальным**. Другими словами, ни один класс не задается раз и навсегда. Поведение каждого класса может быть изменено в любое время любым из его подклассов. Это возможно благодаря **полиморфизму**.

Посмотрите как использовать полиморфизм для расширения гибкости класса.

```
import time

class TrackedVehicle:

    def control_track(self, stop):
        pass

    def turn(self):
        self.control_track(self, True)
        time.sleep(0.25)
        self.control_track(self, False)

class WheeledVehicle:
    def turn_front_wheels(self, on):
        pass

    def turn(self):
        self.turn_front_wheels(self, True)
        time.sleep(0.25)
        self.turn_front_wheels(self, False)
```

Мы определили два отдельных класса, способных производить два разных типа наземных транспортных средств. Основное различие между ними заключается в том, как они поворачивают. Колесный автомобиль обычно просто поворачивает передние колеса. Транспорт на гусеничном ходу должен остановить одну из гусениц.

- чтобы транспорт на гусеничном ходу повернулся, он останавливается и двигается дальше на одной из своих гусениц (за это отвечает метод `controltrack()`, который будет реализован позже)
- колесный автомобиль поворачивает тогда, когда поворачиваются его передние колеса (за это отвечает метод `turnfrontwheels()`)
- метод `turn()` использует метод, подходящий для каждого конкретного транспортного средства.

Что не так с кодом?

Методы `turn()` выглядят слишком похожими, поэтому их нужно изменить.



Построение иерархии классов

Перестроим код следующим образом — введем суперкласс, где соберем все общие схемы управления транспортными средствами, и перенесем все особые случаи в подклассы. Посмотрите на код еще раз.

```
import time

class Vehicle:
    def change_direction(self, on):
        pass

    def turn(self):
        self.change_direction(self, True)
        time.sleep(0.25)
        self.change_direction(self, False)

class TrackedVehicle(Vehicle):
    def control_track(self, stop):
        pass

    def change_direction(self, on):
        self.control_track(self, on)

class WheeledVehicle(Vehicle):
    def turn_front_wheels(self, on):
        pass

    def change_direction(self, on):
        self.turn_front_wheels(self, on)
```

Вот что мы сделали:

- определили суперкласс с именем `Vehicle`, который использует метод `turn()` для реализации общей схемы поворота, а сам поворот выполняется методом `changedirection()`. Примечание: первый метод пуст, так как мы собираемся поместить все детали в подкласс (такой метод часто называют абстрактным методом, поскольку он только демонстрирует некоторую возможность, которая будет реализована позже);
- мы определили подкласс `TrackedVehicle` (примечание: он является производным от класса `Vehicle`), который инстанцировал метод `changedirection()`, используя конкретный метод с именем `controltrack()`;
- соответственно, подкласс `WheeledVehicle` делает то же самое, но использует метод `turnfrontwheel()`, чтобы заставить транспортное средство повернуть.

Самое важное преимущество (кроме читабельности) состоит в том, что такой код позволяет вам реализовать новый алгоритм поворота, просто изменив метод `turn()`. Это можно сделать только в одном месте, а все транспортные средства будут ему подчиняться.

Вот так полиморфизм помогает разработчику поддерживать чистоту и последовательность в коде.



Построение иерархии классов - композиция

Наследование — не единственный способ создания адаптируемых классов. Тех же целей можно достичь (не всегда, но очень часто), используя технику под названием композиция.

Композиция — это процесс создания объекта при помощи других объектов. Объекты, используемые в композиции, предоставляют набор желаемых характеристик (свойств и/или методов), поэтому мы можем сказать, что они действуют как блоки, которые потом используются для построения более сложной структуры.

Можно сказать, что:

- наследование расширяет возможности класса путем добавления новых компонентов и изменения существующих; другими словами, полный рецепт содержится внутри самого класса и всех его предков; объект берет и использует все, что принадлежит классу;
- композиция проецирует класс как контейнер, который может хранить и использовать другие объекты (производные от других классов), где каждый из объектов реализует часть желаемого поведения класса.

Далее мы увидим разницу с помощью ранее определенных транспортных средств. Предыдущий подход привел нас к иерархии классов, в которой самый верхний класс знал об общих правилах, используемых при развороте транспортного средства, но не знал, как управлять соответствующими компонентами (колесами или гусеницами).

Эту способность (управление ТС) реализовали подклассы при помощи специальных механизмов. Сделаем почти то же самое, но с использованием композиции. Как и в предыдущем примере, класс знает, как повернуть транспортное средство, но фактический поворот выполняется специальным объектом, который хранится в свойстве под названием controller. Controller может управлять транспортным средством, манипулируя его деталями.



Построение иерархии классов - КОМПОЗИЦИЯ

Разберемся на примере:

```
import time

class Tracks:
    def change_direction(self, left, on):
        print("tracks: ", left, on)

class Wheels:
    def change_direction(self, left, on):
        print("wheels: ", left, on)

class Vehicle:
    def __init__(self, controller):
        self.controller = controller

    def turn(self, left):
        self.controller.change_direction(left, True)
        time.sleep(0.25)
        self.controller.change_direction(left, False)

wheeled = Vehicle(Wheels())
tracked = Vehicle(Tracks())
wheeled.turn(True)
tracked.turn(False)
```

>>>

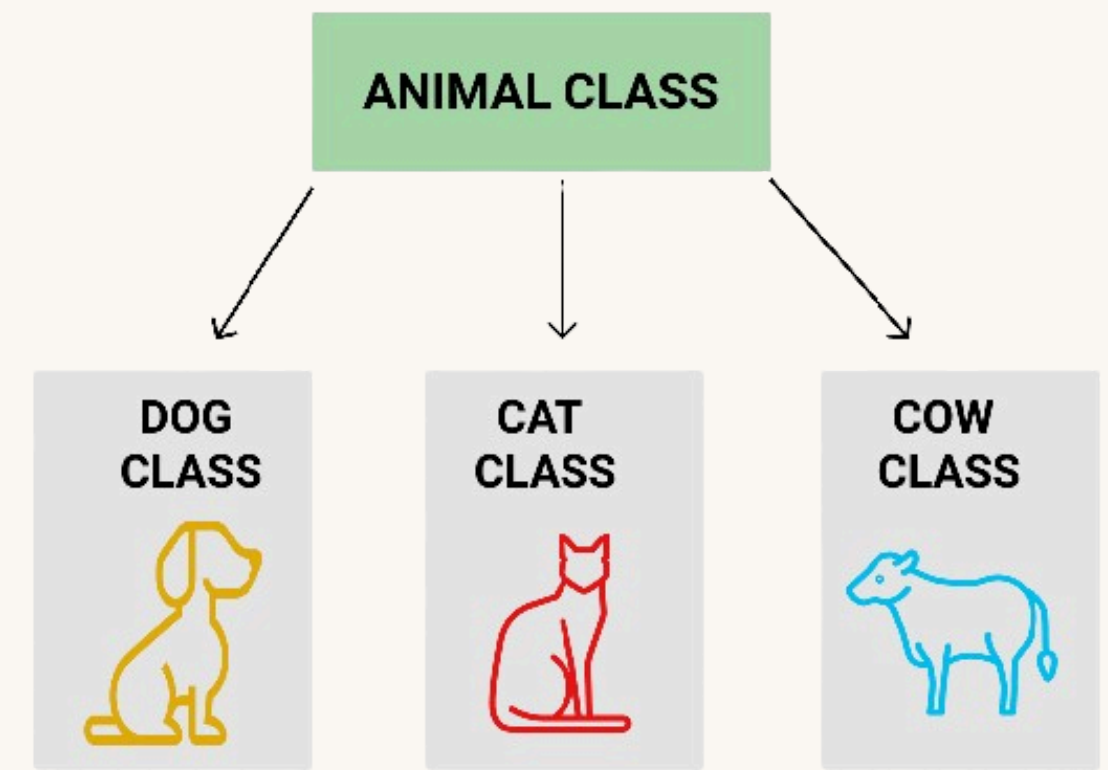
```
wheels:  True True
wheels:  True False
tracks:  False True
tracks:  False False
```

Есть два класса: **Tracks** и **Wheels**. Они знают, как контролировать направление движения транспорта. У нас также есть класс **Vehicle**, который может использовать любой из доступных контроллеров (два уже определенных или любой другой, который будет определен позже) — сам **controller** передается классу во время инициализации.

Таким образом, способность транспортного средства поворачивать состоит из внешнего объекта, не реализованного внутри класса **Vehicle**.

Другими словами, у нас есть универсальный автомобиль, и мы можем установить на него либо гусеницы, либо колеса

Единичное и множественное наследование





Множественное наследование

Вы уже знаете, вам ничто не может помешать использовать множественное наследование в Python. Вы можете получить любой новый класс из нескольких ранее определенных классов.

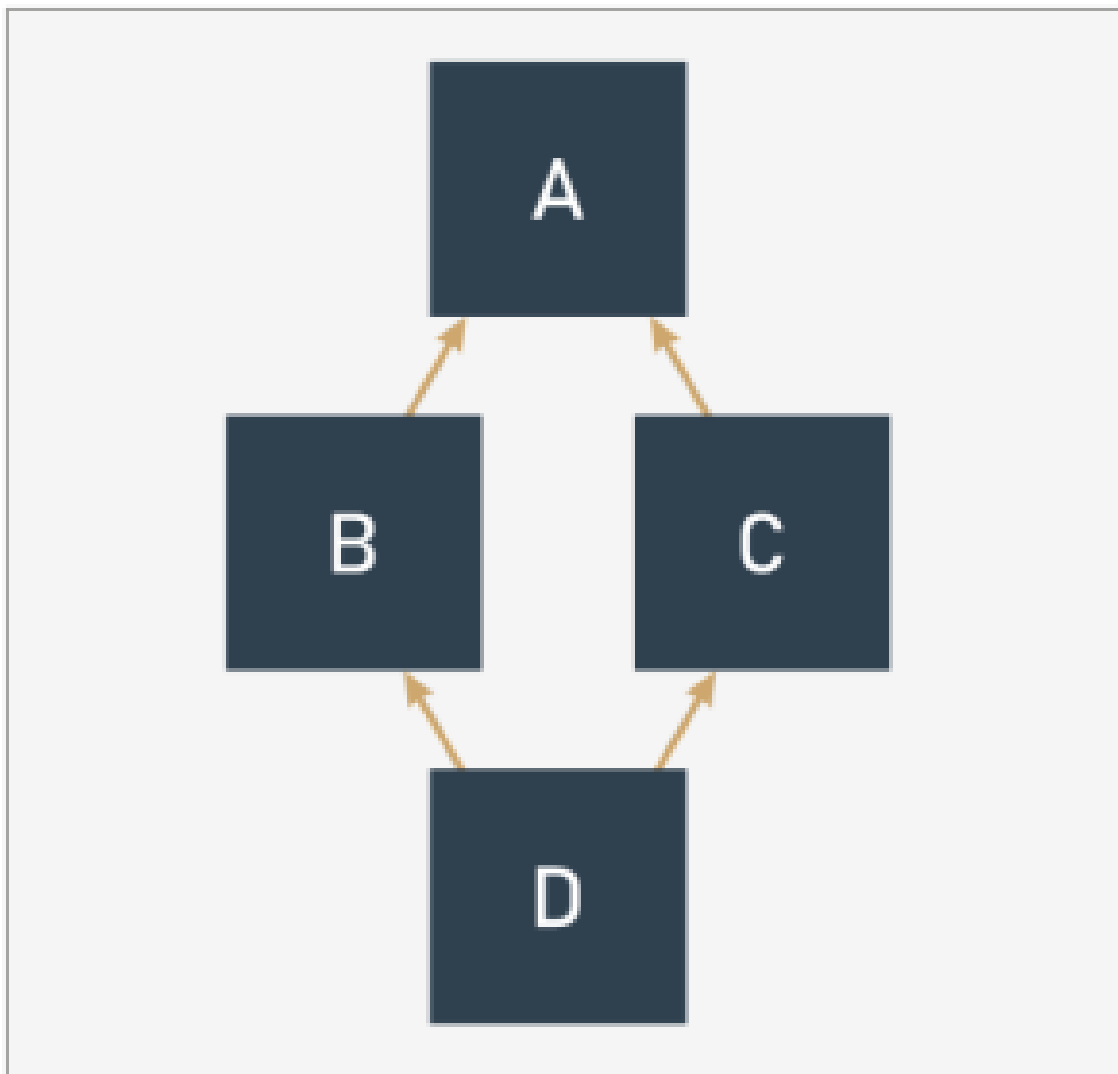
Но есть одно «но». Вы можете, но далеко не всегда должны это делать. Не забывайте, что:

- единичное наследование всегда проще, безопаснее и легче для понимания и использования;
- множественное наследование всегда рискованно, так как тут гораздо больше возможностей ошибиться при определении частей суперклассов, что обязательно повлияет на новый класс;
- переопределение при множественном наследовании может оказаться чрезвычайно сложным; более того, функция `super()` становится неоднозначной;
- множественное наследование нарушает принцип единственной ответственности (подробнее по ссылке: https://ru.wikipedia.org/wiki/Принцип_единственной_ответственности), поскольку новый класс создается из двух (или более) классов, которые ничего не знают друг о друге;
- я настоятельно рекомендую использовать множественное наследование только в качестве крайней необходимости. Если вам нужно много разных функций, принадлежащих разным классам, скорее всего композиция будет лучшим решением.



Ромбы и почему они не нужны

Вопросы, которые могут возникнуть из-за множественного наследования, иллюстрирует классическая «проблема ромба» (diamond problem) или «ромбовидное наследование». Эта проблема получила свое название благодаря очертаниям диаграммы наследования классов в этой ситуации. Посмотрите на рисунок.



У нас есть:

- самый верхний суперкласс A;
- два подкласса, полученные из A — B и C;
- и еще самый нижний подкласс D, полученный из B и C (или C и B, так как в Python эти два варианта будут отличаться)



Ромбы и почему они не нужны

Но Python не любит ромбы и не позволит вам реализовать что-либо подобное. Если вы попытаетесь построить иерархию вроде этой:

```
class A:
    def __init__(self):
        print("class - A")

class B(A):
    def __init__(self):
        print("class - B from:")
        super().__init__()

class C(A):
    def __init__(self):
        print("class - C from:")
        super().__init__()

class D(A, B):
    def __init__(self):
        print("class - D from:")
        super().__init__()

d = D()
```

▼ ▼ ▼

TypeError: Cannot create a consistent method resolution order (MRO) for bases A, B

Однако, если все же без этого не обойтись то можно сделать так:

```
class A:
    def __init__(self):
        print("class - A")

class B(A):
    def __init__(self):
        print("class - B from:", end=' ')
        super().__init__()

class C(A):
    def __init__(self):
        print("class - C from:", end=' ')
        super().__init__()

class D(C, B):
    def __init__(self):
        print("class - D from:", end=' ')
        super().__init__()

d = D()
```

▼ ▼ ▼

class - D from: class - C from: class - B from: class - A

Обратите внимание на порядок наследования и на полученный результат.